

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

Aim: Decision Tree Learning

a) Implement the Decision Tree Learning algorithm to build a decision tree for a given dataset.

b) Evaluate the accuracy and effectiveness of the decision tree on test data.

c) Visualize and interpret the generated decision tree.

Input

```
decision tree T103.py - C:/Users/DELL/Downloads/decision tree T103.py (3.13.11)
File Edit Format Run Options Window Help

# Decision Tree Learning
# AI Practical

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# 1. Load Dataset

data = pd.read_csv("Indian_Student_Placement_Dataset_2025.csv")

print("First 5 records:")
print(data.head())

print("\nDataset Information:")
print(data.info())

print("\nDataset Columns:")
print(data.columns)

# 2. Remove unnecessary ID column

if "Student_ID" in data.columns:
    data = data.drop("Student_ID", axis=1)

if "student_id" in data.columns:
    data = data.drop("student_id", axis=1)

# 3. Display missing values
```

```
*decision tree T103.py - C:/Users/DELL/Downloads/decision tree T103.py (3.13.11)*
File Edit Format Run Options Window Help

# 3. Display missing values

print("\nMissing Values:")
print(data.isnull().sum())

# 4. Convert categorical columns into numbers
data = data.drop(["company_type", "package_ipa"], axis=1)

# 5. Convert categorical columns into numbers

from sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()

for column in data.columns:
    if data[column].dtype == "object":
        data[column] = encoder.fit_transform(data[column].astype(str))

# 6. Set Target Column

target = "placed"

# 7. Separate Features and Target

X = data.drop(target, axis=1)
y = data[target]

print("\nFeatures:")
print(X.columns)

print("\nTarget:")
print(target)

# 8. Split Dataset

X_train, X_test, y_train, y_test = train_test_split(X, y,
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
decision tree T103.py - C:/Users/DELL/Downloads/decision tree T103.py (3.13.11)
File Edit Format Run Options Window Help

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining records:", len(X_train))
print("\nTesting records:", len(X_test))

# 9. Create Decision Tree

model = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=5,
    random_state=42
)

# 10. Train Model

model.fit(X_train, y_train)

print("\nDecision Tree successfully trained.")

# 11. Prediction

y_pred = model.predict(X_test)

print("\nPredicted Values:")
print(y_pred[20])

# 12. Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nDecision Tree Accuracy")

Ln: 104 Col: 14

# 12. Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nDecision Tree Accuracy:")
print(round(accuracy * 100, 2), "%")

# 13. Confusion Matrix

cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# 14. Classification Report

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# 15. Visualize Decision Tree

plt.figure(figsize=(20, 10))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=["Not Placed", "Placed"],
    filled=True,
    rounded=True,
    fontsize=9
)

plt.title("Decision Tree - Student Placement Prediction")

plt.show()

Ln: 124 Col: 0
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

OUTPUT

```
*IDLE Shell 3.13.11*
File Edit Shell Debug Options Window Help

First 5 records:
student_id gender age degree ... projects placed company_type package_lpa
0 1 Male 20 BE ... 0 0 NaN 0.0
1 2 Female 20 BTech ... 4 0 NaN 0.0
2 3 Male 22 BCA ... 1 0 NaN 0.0
3 4 Male 24 BCA ... 4 0 NaN 0.0
4 5 Male 24 BCA ... 4 0 NaN 0.0

[5 rows x 16 columns]

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12000 entries, 0 to 11999
Data columns (total 16 columns):
# Column Non-Null Count Dtype
---
0 student_id 12000 non-null int64
1 gender 12000 non-null object
2 age 12000 non-null int64
3 degree 12000 non-null object
4 branch 12000 non-null object
5 cgpa 12000 non-null float64
6 backlogs 12000 non-null int64
7 internships 12000 non-null int64
8 certifications 12000 non-null int64
9 coding_skills 12000 non-null int64
10 communication_skills 12000 non-null int64
11 aptitude_score 12000 non-null int64
12 projects 12000 non-null int64
13 placed 12000 non-null int64
14 company_type 3652 non-null object
15 package_lpa 12000 non-null float64
dtypes: float64(2), int64(10), object(4)
memory usage: 1.5+ MB
None

Dataset Columns:
Ln: 100 Col: 0

*IDLE Shell 3.13.11*
File Edit Shell Debug Options Window Help

Dataset Columns:
Index(['student_id', 'gender', 'age', 'degree', 'branch', 'cgpa', 'backlogs',
       'internships', 'certifications', 'coding_skills',
       'communication_skills', 'aptitude_score', 'projects', 'placed',
       'company_type', 'package_lpa'],
      dtype='object')

Missing Values:
gender 0
age 0
degree 0
branch 0
cgpa 0
backlogs 0
internships 0
certifications 0
coding_skills 0
communication_skills 0
aptitude_score 0
projects 0
placed 0
company_type 8348
package_lpa 0
dtype: int64

Features:
Index(['gender', 'age', 'degree', 'branch', 'cgpa', 'backlogs', 'internships',
       'certifications', 'coding_skills', 'communication_skills',
       'aptitude_score', 'projects'],
      dtype='object')

Target:
placed

Training records: 9600
Testing records: 2400
Ln: 100 Co
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
"IDLE Shell 3.13.11"
File Edit Shell Debug Options Window Help
dtype: int64

Features:
Index(['gender', 'age', 'degree', 'branch', 'cgpa', 'backlogs', 'internships',
      'certifications', 'coding_skills', 'communication_skills',
      'aptitude_score', 'projects'],
      dtype=object)

Target:
placed

Training records: 9600
Testing records: 2400

Decision Tree successfully trained.

Predicted Values:
[0 0 1 0 1 1 0 0 0 0 0 1 0 0 1 0 1 0]

Decision Tree Accuracy:
100.0 %

Confusion Matrix:
[[1670  0]
 [ 0 730]]

Classification Report:
      precision    recall  f1-score   support

     0       1.00      1.00      1.00      1670
     1       1.00      1.00      1.00       730

 accuracy: 1.00      1.00      1.00      2400
 macro avg: 1.00      1.00      1.00      2400
weighted avg: 1.00      1.00      1.00      2400
```

